SECURITY INFORMATION AND EVENT MANAGEMENT

WAZUH SIEM

Implementation & Configuration Report

Multi-Source Log Collection, Custom Detection Rules & Real-Time SOC Dashboard

1. Executive Summary

This report documents the end-to-end implementation of a Wazuh-based Security Information and Event Management (SIEM) system in a virtualized home lab. The project extends an existing VAPT (Vulnerability Assessment and Penetration Testing) lab with a blue-team layer: the same machines that were attacked in the offensive writeups are now monitored, so every exploit run can be observed from the defender's side.

A Wazuh manager VM collects logs from three sources: a Metasploitable2 target and a Kali Linux attacker (both via rsyslog), and a Windows 11 host (via the Wazuh agent). Custom detection rules were written for SSH brute force, account compromise after repeated failures, and sudo privilege escalation, and a real-time dashboard surfaces high-severity alerts automatically.

Key Achievements

- Deployed a Wazuh manager (OVA) and stabilised it at 4 GB RAM after diagnosing repeated startup failures
- Forwarded all system logs from Metasploitable2 and Kali Linux to Wazuh using rsyslog over UDP/514
- Deployed the Wazuh agent on a Windows 11 host and detected failed logons (Event ID 4625) in real time
- Wrote custom rules (IDs 100010–100016) for brute force, post-brute-force login, and sudo privilege escalation
- Validated detections end-to-end using real attacks from Kali, with MITRE ATT&CK tagging (T1110.001) visible in alerts
- Built a real-time dashboard (10-second refresh, rule.level ≥ 10) so critical alerts surface without manual searching
- Captured exploit traffic with tcpdump to correlate network evidence with SIEM events
- Created a dashboard anomaly detector (1-minute interval) that flags unusual bursts by rule group across authentication, SSH and Windows events

2. Introduction

2.1 What is a SIEM?

A Security Information and Event Management (SIEM) system is a core component of modern security operations. It collects log data from endpoints, servers, network devices and applications, normalises it, correlates events against detection rules, and gives analysts a single place to investigate alerts and meet compliance requirements.

2.2 Why Wazuh?

Wazuh is a widely used open-source security platform combining SIEM and XDR capabilities. Unlike a bare log-search stack, it ships with a large built-in ruleset, decoders for common log formats, and

MITRE ATT&CK mapping, which lets an analyst focus on detection logic rather than building every parser from scratch.

- **Wazuh Manager** — receives events, decodes them, and evaluates them against rules to generate alerts
- **Wazuh Indexer** — stores and indexes alerts and events for search
- **Wazuh Dashboard** — web interface for Threat Hunting, rules management, and visualisation
- **Wazuh Agent** — lightweight endpoint agent (used here on the Windows host)
- **Syslog input** — lets the manager accept logs from devices that cannot run an agent, such as the legacy Metasploitable2

2.3 Log Collection Architecture

Three log sources feed a single manager. Linux machines forward via rsyslog; Windows uses the native agent. The manager decodes each event, applies built-in and custom rules, and exposes the results in the dashboard.

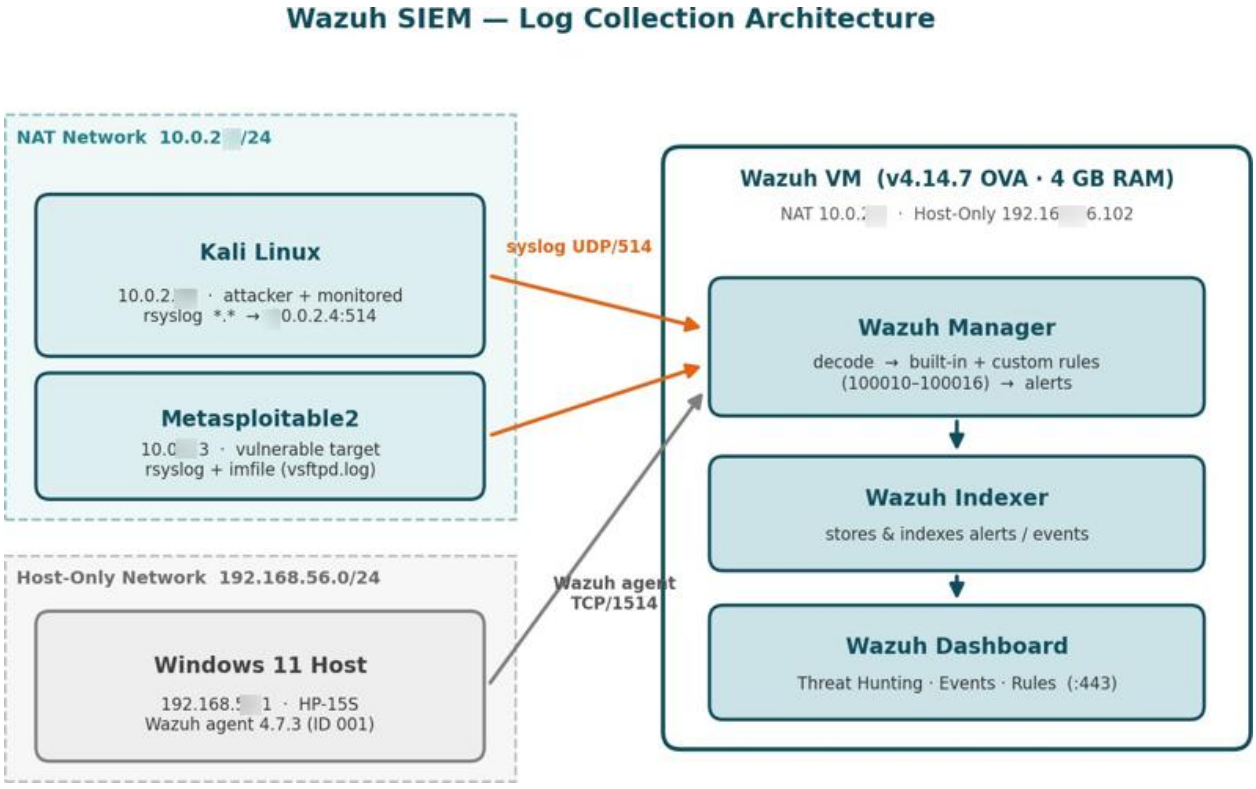


Figure 1: Wazuh SIEM log collection architecture

3. Lab Environment

3.1 Virtual Machine Configuration

The lab runs on Oracle VirtualBox on a Windows 11 host (HP-15S). A shared NAT Network connects the three VMs; a Host-Only adapter on the Wazuh VM additionally lets the Windows host reach the manager.

System	Operating System	Role	IP Address
Wazuh VM	Wazuh v4.14.7 OVA (4 GB RAM, 4 vCPU)	SIEM manager, indexer, dashboard	10.0.2.xxx (NAT) · 192.168.56.xxx (Host-Only)
Kali Linux VM	Kali Linux (Rolling)	Attacker + monitored endpoint	10.0.2.xxx
Metasploitable2 VM	Ubuntu 8.04 (Hardy)	Intentionally vulnerable target	10.0.2.xxx
Windows 11 Host	Windows 11 (HP-15S)	Monitored endpoint (Wazuh agent ID 001)	192.168.56.xxx (Host-Only)

3.2 Network Architecture

- **NAT Network (10.0.2.xxx/24):** Inter-VM traffic. Kali and Metasploitable2 forward syslog to the Wazuh VM at 10.0.2.xxx. This subnet is isolated from the Windows host.
- **Host-Only (192.168.56.xxx /24):** Added to the Wazuh VM so the Windows host can reach the manager (agent registration, dashboard access).

3.3 Software Stack Summary

Component	Version	Host	Port	Protocol
Wazuh manager / indexer / dashboard	4.14.7 (OVA)	Wazuh VM	443	HTTPS (dashboard)
Wazuh agent communication	—	Wazuh VM	1514	TCP (agents)
Syslog receiver	—	Wazuh VM	514	UDP (rsyslog)
rsyslog	Distro default	Kali, Metasploitable2	—	UDP client
Wazuh agent	4.7.3	Windows 11 host	—	Client

4. Installation & Configuration

4.1 Wazuh Manager (OVA)

The Wazuh manager was deployed from the official pre-built Wazuh v4.14.7 VirtualBox OVA, which bundles the manager, indexer and dashboard on a single VM. The VM was configured with 4 GB RAM, 4 vCPUs and a 25 GB disk, with two adapters (NAT Network and Host-Only), from first boot; 3 GB proved insufficient (see Section 8.1).

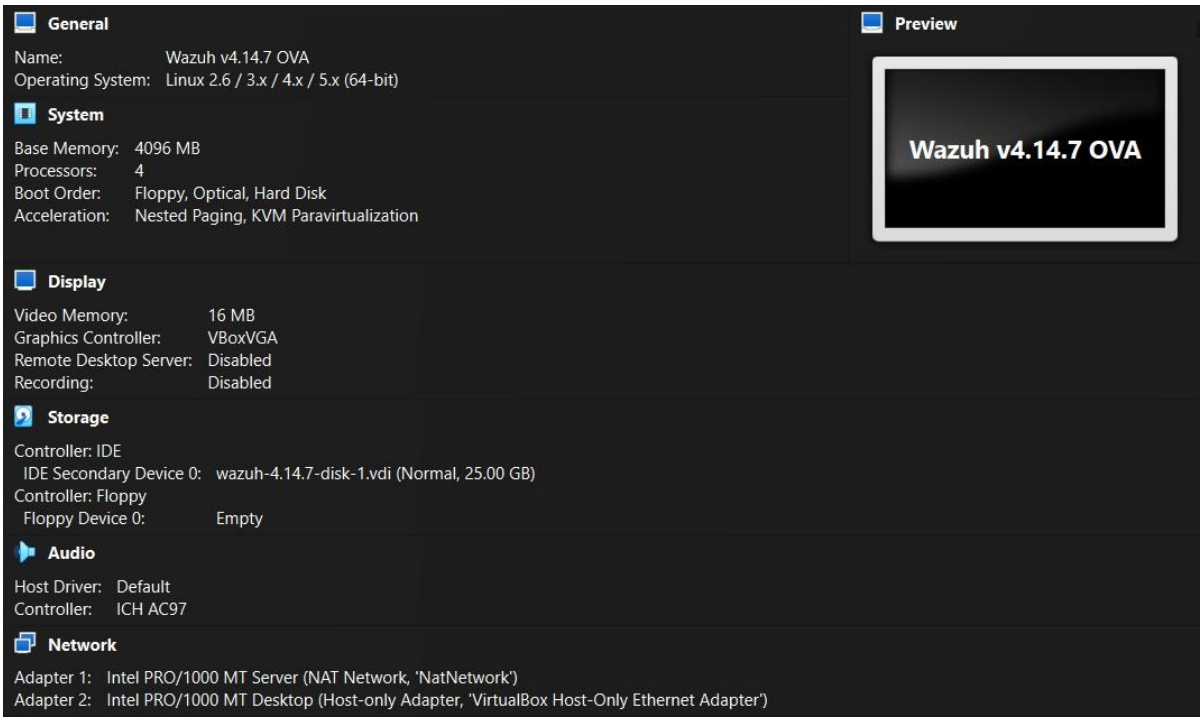


Figure 2: Wazuh v4.14.7 OVA VM settings in VirtualBox (4096 MB RAM, 4 CPUs, NAT Network + Host-Only adapters)

4.2 Enabling Syslog Reception

By default the manager listens only for secure Wazuh-agent traffic on TCP/1514. To accept plain syslog from Kali and Metasploitable2, an additional <remote> block was added to /var/ossec/etc/ossec.conf:

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>10.0.2.xxx  /24</allowed-ips>
</remote>
```

The allowed-ips restriction limits syslog acceptance to the lab subnet only.

4.3 Enabling Full Log Archiving

Wazuh only writes alerts by default. To keep every received event (useful for verifying the pipeline and for analysing traffic that does not trigger a rule), full archiving was enabled in ossec.conf:

```
<logall>yes</logall>
```

Events are then written to /var/ossec/logs/archives/archives.log.

4.4 rsyslog on Metasploitable2

Metasploitable2 runs Ubuntu 8.04, which is end-of-life and cannot run a modern Wazuh agent. rsyslog was installed instead, after repointing the dead apt repositories in /etc/apt/sources.list to old-releases.ubuntu.com. All logs are forwarded to the manager with a wildcard rule, and the imfile module natively tails the vsftpd log so FTP activity is captured alongside system logs.

```
# /etc/rsyslog.conf (forwarding)
*.* @10.0.2.xxx :514
```

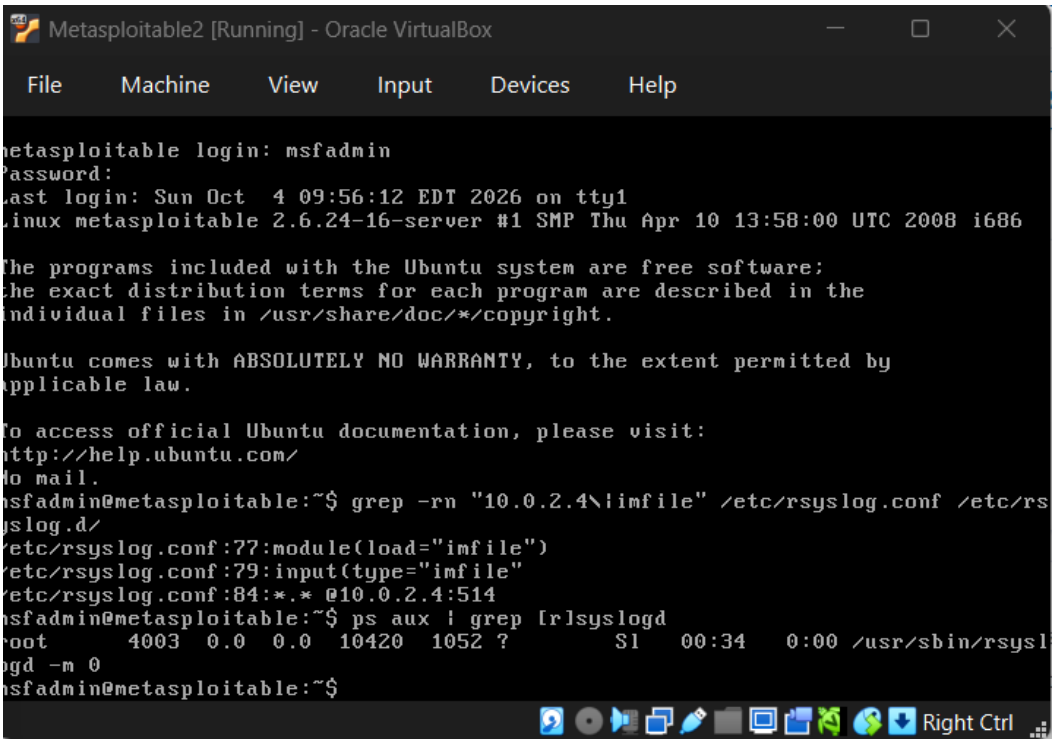


Figure 3: Metasploitable2 rsyslog forwarding rule (@10.0.2.xxx:514), imfile module, and rsyslogd running

4.5 rsyslog on Kali Linux

Kali was configured with the same wildcard forwarding rule so that attacker-side activity (commands run, network events) is also visible in the SIEM. Both rsyslog services restart automatically on reboot, removing the need for manual log-forwarding commands.

```
*.* @10.0.2.xxx :514
```

4.6 Wazuh Agent on Windows 11

The Windows host runs Wazuh agent 4.7.3 (agent ID 001). Because the NAT Network is isolated from the host, a Host-Only adapter (192.168.56.xxx) was added to the Wazuh VM so the agent could reach the manager. The agent runs as a Windows service and starts automatically.

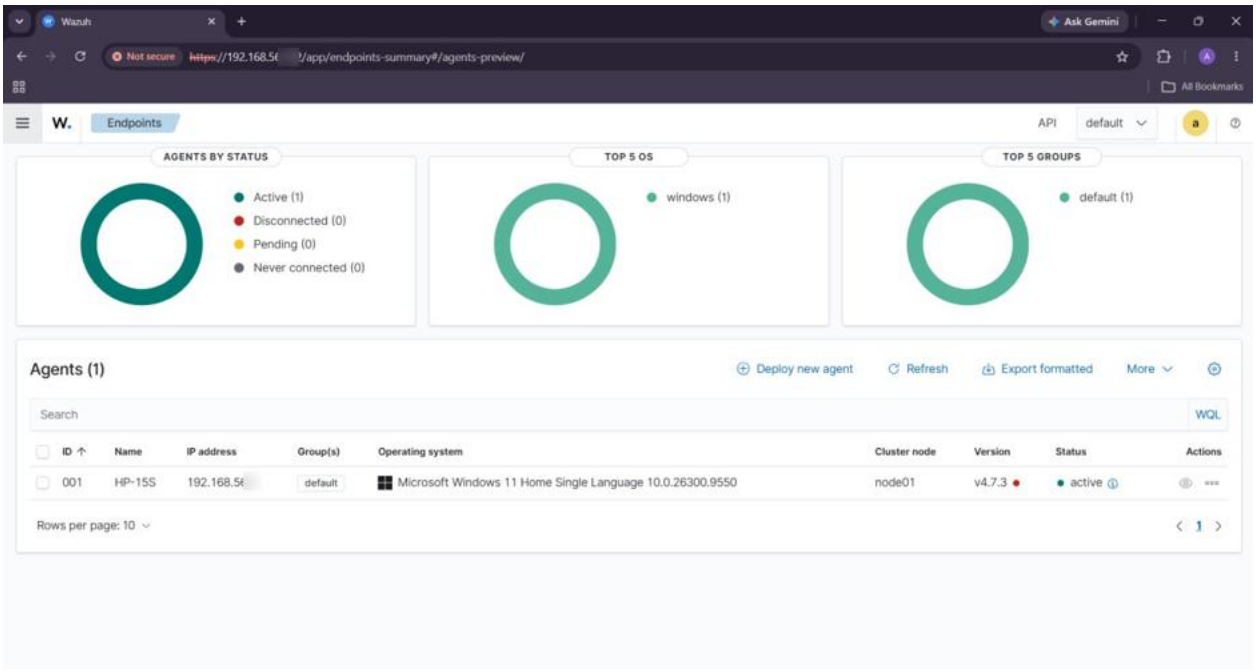


Figure 4: Endpoints summary showing agent 001 (HP-15S, Windows 11, v4.7.3) active

4.7 Dashboard Access

The dashboard is the same instance reached from two networks:

- From Kali / Metasploitable2 network: `https://`10.0.2.xxx
- From the Windows host: `https://`192.168.56.xxx

> **Safe Boot Order**
> Start the Wazuh VM first and wait 3–4 minutes for all services to come up, then start Kali and Metasploitable2. The Windows agent starts automatically as a service.

5. Pipeline Validation

5.1 Verifying Logs Reach the Manager

After configuring both Linux sources, the vsftpd exploit was re-run from Kali against Metasploitable2 and archives.log was inspected on the Wazuh VM. Entries from both 10.0.2.xxx and 10.0.2.xxx were present, confirming the rsyslog pipeline was working end-to-end.

5.2 Splitting Logs by Source

Because both machines write into one archives.log alongside the manager's own internal messages, a small post-processing script (split_logs.py) was written to read the file once and split entries by source IP. This keeps each source easy to inspect on its own.

Output File	Contents	Lines
`metasploitable_logs.log`	Events from 10.0.2.xxx	389
`kali_logs.log`	Events from 10.0.2.xxx	90

Output File	Contents	Lines
other_logs.log	Wazuh internal / other noise	929

5.3 Packet Capture Correlation

To pair network evidence with SIEM events, tcpdump was run on Kali during the vsftpd exploit, filtered to the target host:

```
sudo tcpdump -i eth0 host 10.0.2.xxx -w vsftpd_exploit.pcap
```

The capture completed cleanly (12 packets captured, 0 dropped) and shows the exploit conversation between Kali and Metasploitable2.

5.4 Events in the Dashboard

A custom rule (Section 6) turns incoming log lines into classified alerts. Traffic from both hosts now appears under Threat Hunting → Events in the dashboard, not just as raw lines in archives.log.

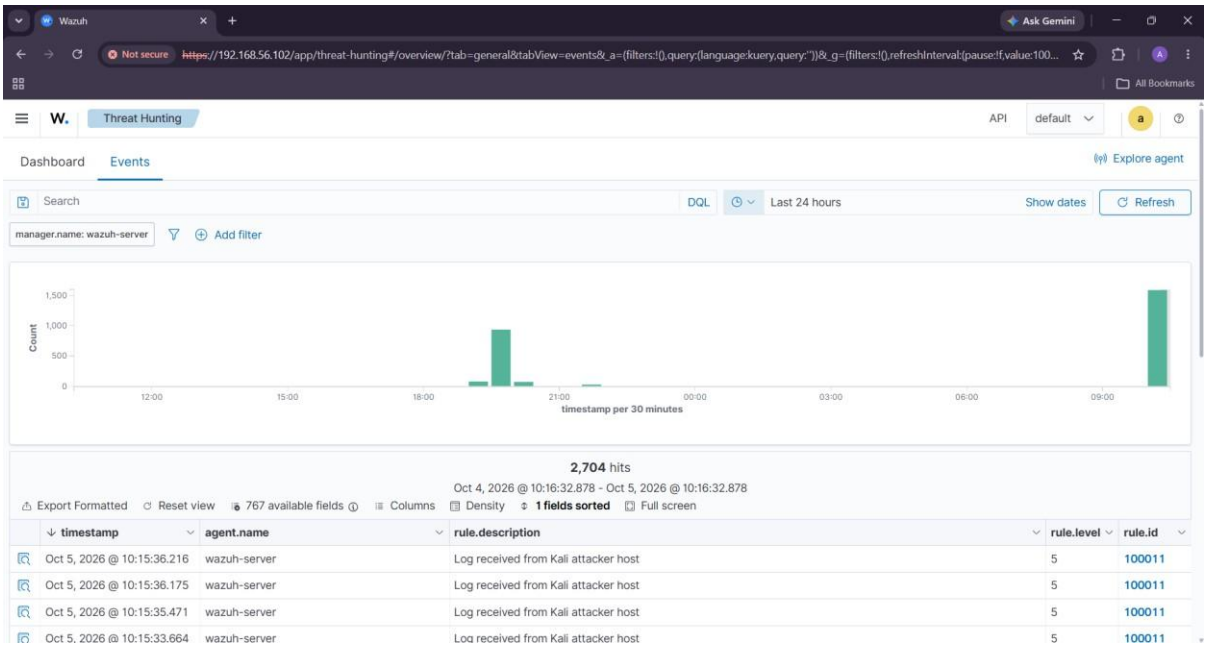


Figure 5: Threat Hunting → Events: rule 100011 "Log received from Kali attacker host" (level 5), 2,704 hits in the last 24 hours

6. Custom Detection Rules

Custom rules live in /var/ossec/etc/rules/local_rules.xml and were created and edited entirely through the dashboard (Management → Rules → Manage rules files → local_rules.xml). After saving, the manager reloaded the ruleset successfully.

6.1 Rule Summary

Rule ID	Level	Purpose	Logic
100010 / 100011	—	Generic "Log received from X" alert for Metasploitable2 / Kali	Matches on <hostname>
100013	10	SSH brute force: 5 failed logins in 2 minutes	Frequency 5 / timeframe 120 on rule 5760, same source IP
100014	12	Successful SSH login after multiple failures (possible account compromise)	Builds on 5760 + 5715, same source IP
100015	12	Sudo to a root shell (privilege escalation)	Builds on 5402, matches COMMAND for /bin/bash, sh, su
100016	10	Any sudo to root	Builds on 5402 and 5403

6.2 Brute-Force Rule

Rule 100013 is built on top of Wazuh's built-in rule 5760 (sshd authentication failed), which already extracts the source IP and target user and carries a MITRE T1110.001 (Password Guessing) tag. Representative form:

```
<rule id="100013" level="10" frequency="5" timeframe="120">
  <if_matched_sid>5760</if_matched_sid>
  <same_source_ip />
  <description>SSH brute force: 5 failed logins in 2 minutes</description>
</rule>
```

6.3 Testing with the Ruleset Test Tool

Rules were debugged using the dashboard's built-in Ruleset Test (Management → Rules → Ruleset Test). Pasting a sample log line shows exactly which decoder and rule matched at each phase, which made it possible to diagnose rule-ordering problems without touching the CLI.

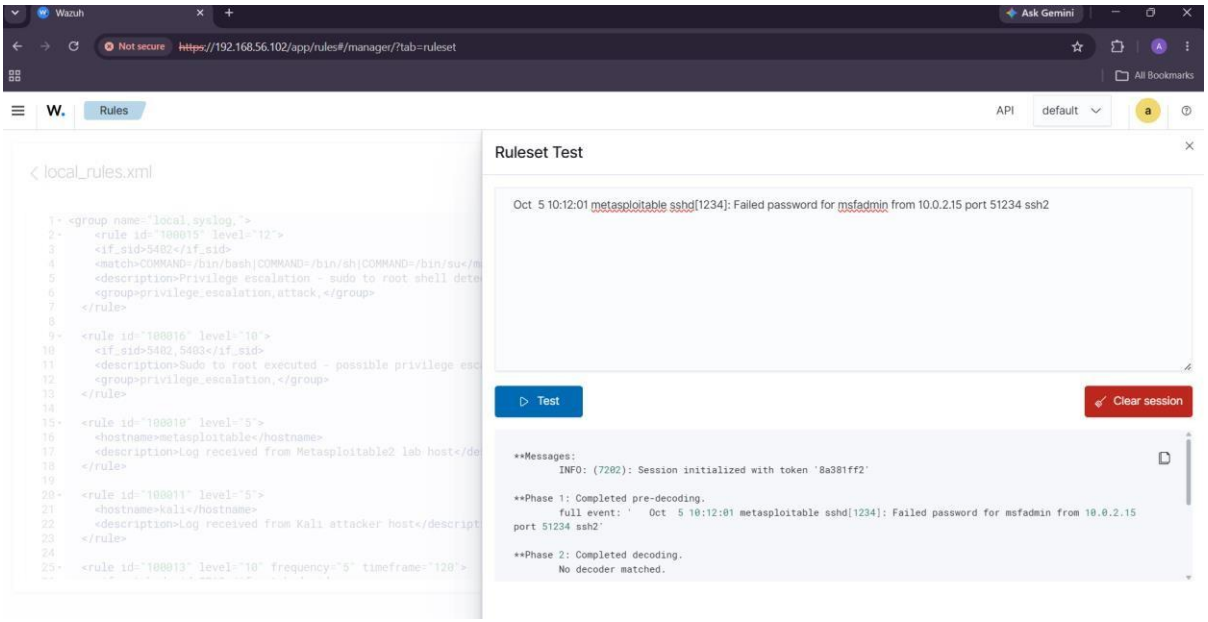


Figure 6: Ruleset Test (Management → Rules) with local_rules.xml behind it; phase 2 reports no decoder matched for this pasted sample line

7. Attack Simulation & Detection

Detections were validated by launching real attacks from Kali against the monitored targets and confirming the expected alerts in the dashboard.

Scenario	Source → Target	Expected Detection	Result
vsftpd 2.3.4 backdoor exploit	Kali → Metasploitable2	FTP log lines received and classified in Events	✓Pass
Repeated failed SSH logins	Kali → Metasploitable2	Rule 5760, then 100013 (level 10)	✓Pass
Successful login after failures	Kali → Metasploitable2	Rule 100014 (level 12)	✓Pass
Sudo to root / root shell	On monitored host	Rules 100016 / 100015	✓Pass
Failed Windows logon	Local attempt on host	Event ID 4625, rule 60204	✓Pass

7.1 SSH Brute Force from Kali

Metasploitable2 only offers legacy ssh-rsa/ssh-dss host keys, which Kali's modern OpenSSH client rejects by default. The failed logins were generated with sshpass and the legacy algorithms explicitly allowed:

```
sshpass -p wrongpass ssh -o HostKeyAlgorithms=+ssh-rsa \
  -o PubkeyAcceptedKeyTypes=+ssh-rsa user@10.0.2.xxx
```

After five failures within two minutes, rule 100013 fired with full decoded fields (srcip, dstuser) and the MITRE tag visible.

8. Challenges & Troubleshooting

This section documents the real problems encountered during the build and how each was diagnosed and resolved.

8.1 Wazuh Manager Crashing Under Load

Problem: The manager timed out or crashed on startup when under load.

Root Cause: The VM had only 3 GB RAM, too little for manager + indexer + dashboard.

Resolution: RAM was raised to 4 GB, which restored stability.

8.2 Repeated Startup Failures After Reboots

Problem: After several VM reboots the Wazuh manager service kept failing on startup, even with 4 GB RAM.

Root Cause: Accumulated bad state from repeated crash/reboot cycles on the original VM — not a resource or network issue.

Resolution: The VM was rebuilt from a fresh OVA import with 4 GB RAM set from first boot. The old VM was kept as a renamed backup. The new VM received the same DHCP IP (10.0.2.xxx), so no forwarding changes were needed on Kali or Metasploitable2.

8.3 Syslog Not Being Accepted

Problem: Kali and Metasploitable2 were sending syslog but nothing arrived on the manager.

Root Cause: The default ossec.conf only listens for Wazuh-agent traffic on TCP/1514, not raw syslog.

Resolution: Added a <remote> block with connection=syslog on UDP/514, restricted to 10.0.2.xxx/24.

8.4 archives.log Empty on the Rebuilt VM

Problem: Received events were not being written to archives.log.

Root Cause: Full archiving (logall) is off by default and the setting did not carry over to the fresh install.

Resolution: Enabled <logall>yes</logall> in ossec.conf and restarted the manager.

8.5 Metasploitable2 Package Installation Failing

Problem: apt could not fetch rsyslog on Metasploitable2.

Root Cause: Ubuntu 8.04 is end-of-life and its repositories no longer exist at the default URLs.

Resolution: Pointed /etc/apt/sources.list at old-releases.ubuntu.com, then installed rsyslog.

8.6 Decoder / Rule Not Matching

Problem: A custom rule written to classify specific log lines never matched; wazuh-logtest never reached the rule-matching phase.

Root Cause: Wazuh's built-in decoders do not cover every daemon's log format (for example dhclient), so those lines were never decoded.

Resolution: Wrote a simple rule keyed on the <hostname> field instead of decoding message content, producing a generic "Log received from X" alert (100010/100011) for each host.

8.7 Custom Brute-Force Rule Never Fired

Problem: Custom rule 100012 (match on "Failed password|authentication failure|Invalid user") never triggered.

Root Cause: Built-in rules claim the sshd failure lines first: 2501 catches the PAM line and 5760 catches "Failed password …", already decoded with srcip and dstuser.

Resolution: Used the dashboard Ruleset Test to see which rule matched, dropped the custom match rule, and built 100013 on top of built-in rule 5760 using frequency/timeframe and same_source_ip.

8.8 SSH Client Rejecting Metasploitable2

Problem: SSH from Kali to Metasploitable2 failed before authentication.

Root Cause: Modern OpenSSH rejects the target's old ssh-rsa/ssh-dss host keys by default.

Resolution: Explicitly enabled +ssh-rsa for host key and public-key algorithms on the client side for the test.

8.9 Windows Host Could Not Reach the Manager

Problem: The Windows agent could not connect, and the dashboard was unreachable from the host.

Root Cause: The NAT Network is isolated from the Windows host.

Resolution: Added a Host-Only adapter (192.168.56.xxx) to the Wazuh VM alongside the NAT adapter.

9. Verification & Testing

9.1 Service & Source Health

Component	Status	Notes
Wazuh manager	✓Active	Stable at 4 GB RAM on rebuilt VM
Wazuh dashboard	✓Active	https://10.0.2.xxx and https://192.168.56.xxx
rsyslog — Metasploitable2	✓Active	Forwarding all logs + vsftpd via imfile
rsyslog — Kali	✓Active	Forwarding all logs, auto-restart on reboot

Component	Status	Notes
Wazuh agent — Windows	✓Active	Agent ID 001, v4.7.3

9.2 End-to-End Data Flow Confirmation

- Kali and Metasploitable2 events observed in archives.log after exploit runs
- Logs correctly split per source (389 / 90 / 929 lines)
- tcpdump captured 12 exploit packets with 0 dropped
- Rules 100010–100016 visible firing in Threat Hunting → Events with decoded fields
- Windows failed logon (Event ID 4625, rule 60204) detected in real time

10. Data Flow Architecture

The path a log event takes from source to analyst:



Figure 10: Log data flow from source to analyst

11. Dashboard & Log Analysis

11.1 Real-Time SOC Dashboard

A dedicated view under Threat Hunting → Dashboard was configured with auto-refresh every 10 seconds and a filter of rule.level >= 10. High-severity alerts (brute force, compromise, privilege escalation) therefore appear on their own without manually searching logs.

Figure 7 shows the Threat Hunting dashboard overview for the last 24 hours, accessed from the Windows host over the Host-Only address.

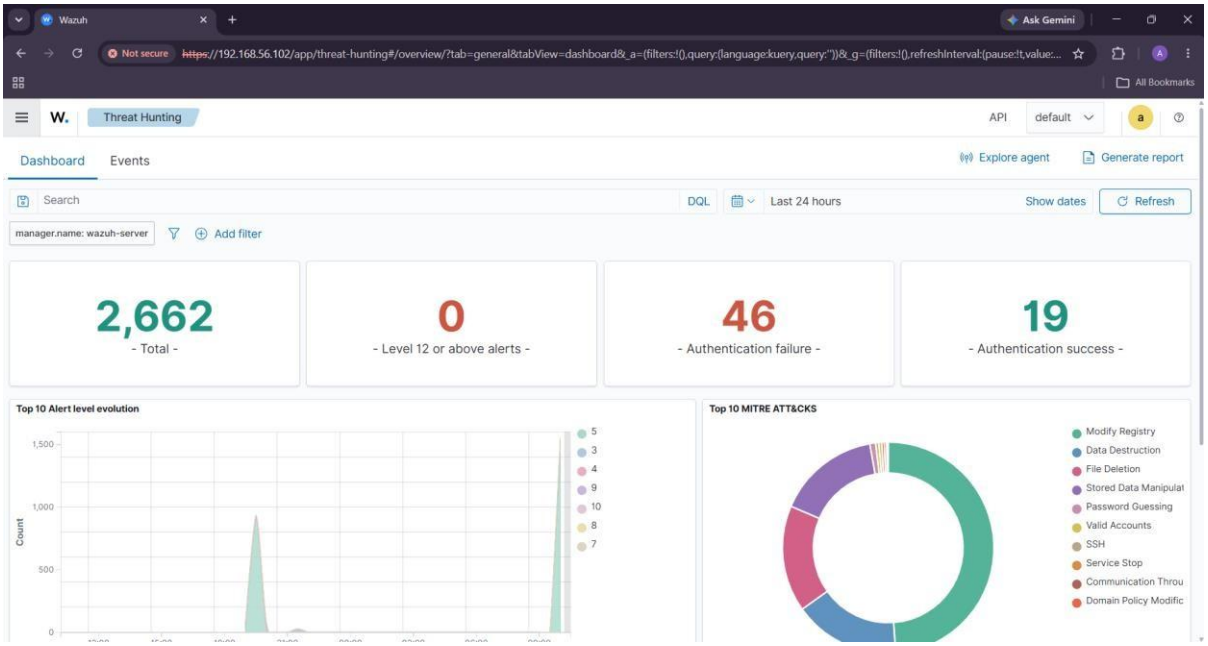


Figure 7: Wazuh Threat Hunting dashboard (last 24 hours): 2,662 total events, 46 authentication failures, 19 authentication successes, MITRE ATT&CK techniques including Password Guessing, Valid Accounts and SSH

11.2 Raw Logs in Discover

While the Threat Hunting views show classified alerts, the Discover page (Explore → Discover, index pattern wazuh-alerts-*) exposes every stored record together with its original log line. This is the place to confirm that a source is actually reaching the manager and to read the evidence behind an alert.

Figure 11 shows the last 24 hours: 3,946 hits, with activity peaks in the evening of 4 October and the late morning of 5 October. Each expanded record carries the decoded fields and the raw message in full_log. The records at the top are dhclient lines forwarded from Metasploitable2 (10.0.2.xxx), for example a permission-denied error when creating a lease file. Because dhclient has no built-in decoder (see Section 8.6), these lines are classified by the generic rule 100010, "Log received from Metasploitable2 lab host" (level 5). The records also show agent.name wazuh-server, manager.name, location (the sender IP), predecoder.hostname and predecoder.program_name.

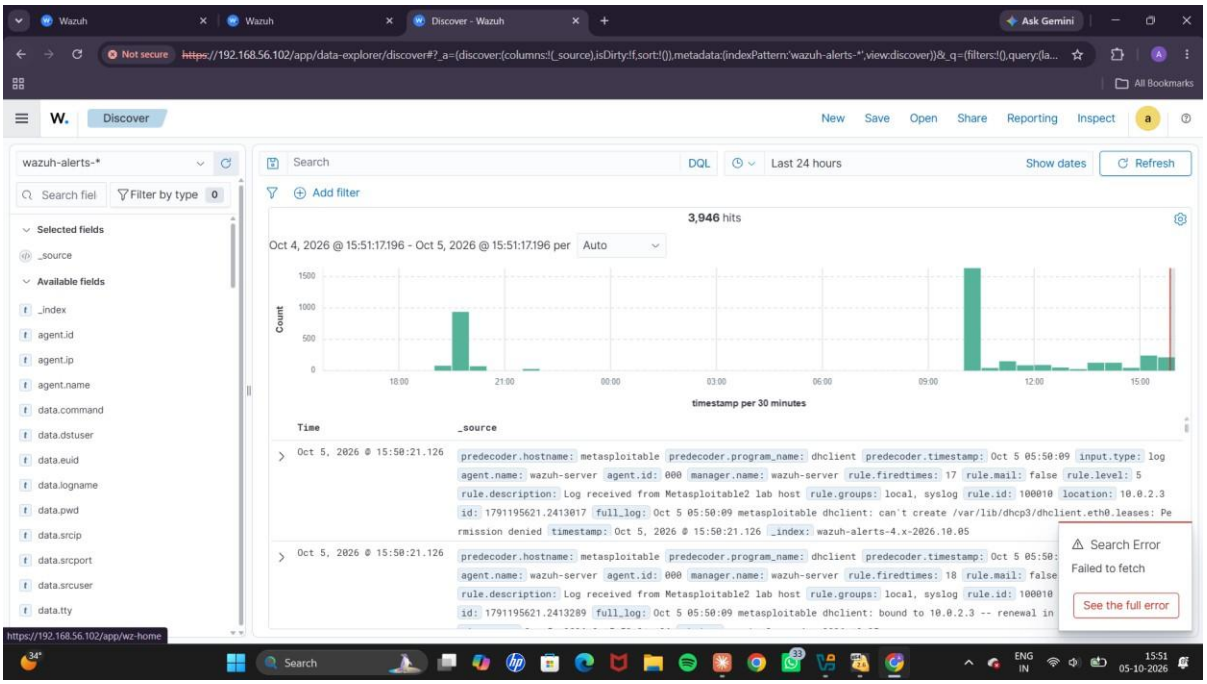


Figure 11: Discover (wazuh-alerts-, last 24 hours): 3,946 hits with raw log lines such as dhclient messages from Metasploitable2 under rule 100010*

11.3 Alerts by Rule in Threat Hunting Events

To separate real detections from background noise, the Events tab under Threat Hunting was queried with rule.level >= 10 (saved query "critical") and a manager.name: wazuh-server filter. Figure 12 shows 40 hits in the last 24 hours, listed with timestamp, agent.name, rule.description, rule.level and rule.id.

The alerts visible in this view map directly to the custom rules in Section 6: rule 100016 (sudo to root executed, possible privilege escalation), rule 100013 (multiple failed SSH authentication attempts from the same source, 5 failures in 2 minutes) and rule 100017 (multiple FTP authentication failures from the same source, possible FTP brute force). All three are level 10. The histogram shows the alerts clustering around the attack-simulation windows rather than being spread evenly, which is the pattern expected when the level filter removes the level 5 "Log received" noise seen in Discover.

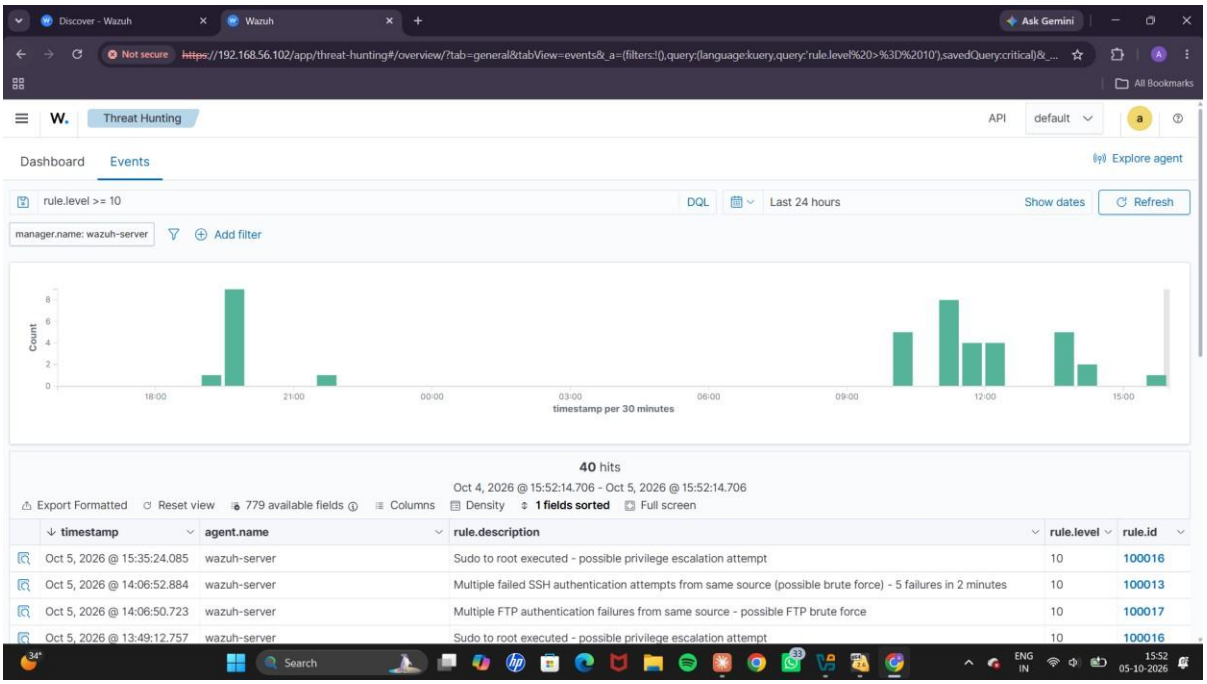


Figure 12: Threat Hunting → Events filtered on rule.level >= 10: 40 hits in the last 24 hours, including rules 100016, 100013 and 100017

11.4 Key Fields for Analysis

Field	Description
`timestamp`	Event time — used for timeline correlation
`agent.name`	Source host that generated the event
`rule.id / rule.level`	Which rule fired and its severity
`rule.description`	Human-readable alert description
`rule.mitre.id`	MITRE ATT&CK technique ID (e.g. T1110.001)
`data.srcip`	Source IP extracted by the decoder
`data.dstuser`	Targeted user account
`full_log`	Raw original log line

11.5 Anomaly Detection

In addition to rule-based detection, an anomaly detector (soc-lab-anomaly-detector) was created in the dashboard's Anomaly Detection plugin to highlight unusual bursts of activity without needing a hand-written rule. It runs on a 1-minute detector interval and has been running continuously since 1 October 2026.

The Real-time results view (Figure 8) plots the anomaly grade for each interval across the last 60 minutes. Anomaly grade runs from 0 (none) to 1 (critical); the panel also reports the model's confidence in that result.

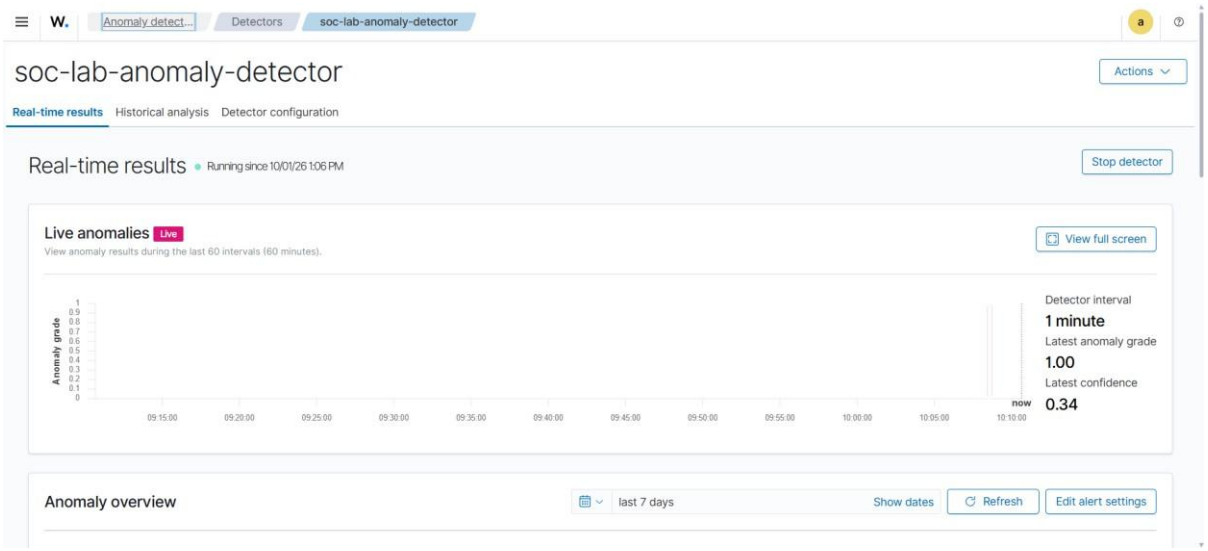


Figure 8: Real-time results for soc-lab-anomaly-detector (1-minute interval, running since 10/01/26)

The Anomaly overview heat map (Figure 9) breaks results down by rule.groups (top 10, by severity) over the last 7 days. It shows which kinds of activity were anomalous and when: authentication_success, authentication_failures, authentication_failed, pam, sshd, policy_changed and the Windows groups (windows, windows_application, windows_security, WEF). Anomalies cluster on 1, 3 and 4 October.



Figure 9: Anomaly overview heat map by rule.groups (top 10, by severity), last 7 days

12. Tool Selection Rationale

Wazuh was chosen over assembling a log-search stack by hand because the goal was SOC-analyst practice — writing and tuning detections and investigating alerts — rather than building parsers from scratch.

Criterion	Wazuh	ELK Stack	Graylog	Security Onion
Built-in detection rules	✓	~ (via Elastic Security)	~	✓
MITRE ATT&CK mapping	✓	~	✗	✓
Endpoint agent included	✓	~ (Beats / Elastic Agent)	✗	✓
Accepts raw syslog	✓	✓	✓	✓
Setup effort	Low (OVA)	Medium	Medium	High
Resource usage	Medium	Medium–High	Medium	High

Key: ✓strong support · ~ partial or requires extra setup · ✗not a core feature. Ratings are a general assessment for a small home-lab use case.

Why Wazuh Was Chosen

- **Detection engineering practice —** writing custom rules on top of built-in ones mirrors real SOC tuning work
- **Works with legacy targets —** syslog input covered Metasploitable2, which cannot run a modern agent
- **Single-VM deployment —** the OVA bundles manager, indexer and dashboard, which suits a limited-RAM host
- **MITRE-tagged alerts —** detections map directly to ATT&CK techniques, useful for writeups

13. Limitations & Future Work

Current Limitations

- Syslog is forwarded over plain UDP/514 without encryption or delivery guarantees — acceptable in an isolated lab, not in production
- The dashboard uses the default admin credentials, appropriate only for a closed lab network
- Rules 100010/100011 are hostname-based and generic; they classify traffic but do not describe what happened
- Only one attack family (SSH authentication) has dedicated correlation rules so far

Planned Improvements

- Add detection rules for the remaining VAPT exploits (UnrealIRCd, Samba, Tomcat)
- Deploy a Cowrie honeypot to generate realistic attack data
- Move syslog to TCP with TLS, and replace default credentials
- Add Wazuh active response and email notifications for level 12 alerts

14. References

1. Wazuh Documentation: https://documentation.wazuh.com/current/index.html
2. Wazuh Ruleset (rules & decoders): https://documentation.wazuh.com/current/user-manual/ruleset/index.html
3. Wazuh Syslog collection: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/index.html
4. rsyslog Documentation: https://www.rsyslog.com/doc/
5. MITRE ATT&CK T1110.001 Password Guessing: https://attack.mitre.org/techniques/T1110/001/
6. Metasploitable 2 (Rapid7): https://docs.rapid7.com/metasploit/metasploitable-2/

15. Conclusion

This project delivered a working Wazuh SIEM in a virtualized home lab, fed by three log sources and tuned with custom detection rules for common attack behaviours. Each stage — manager deployment, syslog and agent collection, rule development, and dashboard monitoring — was validated using real attacks from Kali rather than sample data.

The troubleshooting was as valuable as the final result: RAM sizing, corrupted service state, syslog configuration, decoder gaps, and rule-precedence conflicts are all issues that appear in real SIEM deployments. Resolving them built practical skills directly relevant to SOC analyst work, and the lab now provides a base for further detection engineering and incident-response practice.

— End of Report —